FSMP: A Frontier-Sampling-Mixed Planner for Fast Autonomous Exploration of Complex and Large 3-D Environments

Shiyong Zhang, *Member, IEEE*, Xuebo Zhang, *Senior Member, IEEE*, Qianli Dong, *Graduate Student Member, IEEE*, Ziyu Wang, Haobo Xi, and Jing Yuan, *Member, IEEE*

***Abstract*— In this article, we propose a systematic framework for fast exploration of complex and large 3-D environments using micro aerial vehicles (MAVs). The key insight is the organic integration of the frontier- and sampling-based strategies that can achieve rapid global exploration of the environment. Specifically, a field-of-view (FOV)-based frontier detector with the guarantee of completeness and soundness is devised for identifying 3-D map frontiers. Different from random sampling-based methods, the deterministic sampling technique is employed to build and maintain an incremental road map based on the recorded sensor FOVs and newly detected frontiers. With the resulting road map, we propose a two-stage path planner. First, it quickly computes the global optimal exploration path on the road map using the lazy evaluation strategy. Then, the best exploration path is smoothed to further improve the exploration efficiency. We validate the proposed method both in simulation and real-world experiments. The comparative results demonstrate the promising performance of our planner in terms of exploration efficiency, computational time, and explored volume.**

***Index Terms*— Autonomous exploration, environmental mapping, frontier detection, micro aerial vehicle (MAV), sampling-based algorithm.**

I. Introduction

AUTONOMOUS exploration plays a pivotal role in many robotic applications, such as infrastructure inspection [1], environmental modeling [2], [3], robot collaboration [4], and search and rescue [5], among others. Benefiting from the high maneuverability, micro aerial vehicles (MAVs) have been widely employed for exploring 3-D unknown environments. Nevertheless, the main challenge of autonomous exploration is that the robot is required to make decisions online for navigation in unknown environments, which will become trickier when the employed MAV is resource-limited.

There are two mainstream strategies for addressing the autonomous exploration problem: 1) sampling-based exploration [6], [7], [8], [9] and 2) frontier-based exploration [10], [11], [12], [13], [14]. For the sampling-based strategy, it explores the environment by randomly sampling viewpoints in the free space, which has great potential for exploring an independent region [6]. However, this type of method usually gets stuck in local minima when the environment is complex and consists of many separate regions [8]. Differently, the frontier-based strategy starts by detecting the frontiers of the whole environment and directs the robot to these regions to complete the exploration. Even if this strategy can achieve the global evaluation of the exploration candidates, it usually results in robots going back and forth in the environment. To relieve the problem, the works proposed in [11], [12], and [13] compute the visit sequence of frontiers by solving the traveling salesman problem (TSP), which is known as an NP-hard problem. Therefore, the effort required for solving TSP will increase dramatically with the size of the environment, resulting in a formidable computational overhead for real-time path planning (especially in complex and large 3-D environments). It is intuitive to combine these two strategies in one unified framework [15], [16], [17], [18]. However, the critical problem is how to achieve an effective integration of different strategies. Existing methods generally adopt the sampling-based strategy for local exploration and utilize frontiers for global exploration planning when the local exploration finishes. Practically, such a naive combination is still prone to local minima, leaving room for further improvement.

In this article, we propose a frontier-sampling-mixed exploration planner (FSMP), which can achieve real-time performance onboard an MAV. Specifically, we propose a fast field-of-view (FOV)-based frontier detector (F^3D) that only examines the minimal space every time. The detector can identify frontiers efficiently and accurately when new sensor measurements are received. To achieve global coverage, a road map is incrementally built by using the uniformly deterministic sampling method. With the road map in hand, the path from the current position of the robot to each exploration candidate and its corresponding motion cost can be queried directly, thereby facilitating the evaluation of all exploration candidates in a global context. To further improve the efficiency, a two-stage path planner is devised. First, we propose a lazy evaluation strategy that benefits from the breadth-first search (BFS) on the road map. This strategy allows us to obtain the

Received 25 July 2024; revised 7 October 2024; accepted 1 November 2024. Date of publication 3 March 2025; date of current version 26 March 2025. This work was supported in part by the Natural Science Foundation of China under Grant 62303249 and Grant 62293513/62293510, and in part by China Postdoctoral Science Foundation-Tianjin Joint Support Program under Grant 2023T013TJ. The Associate Editor coordinating the review process was Dr. Lihui Peng. *(Corresponding author: Xuebo Zhang.)*

The authors are with the Institute of Robotics and Automatic Information System, College of Artificial Intelligence, Nankai University, Tianjin 300350, China (e-mail: syzhang@mail.nankai.edu.cn; zhangxuebo@nankai.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TIM.2025.3547488, provided by the authors.

Digital Object Identifier 10.1109/TIM.2025.3547488

global optimal exploration path while avoiding the need to compute the utility of all candidates. Second, we optimize the best exploration path to improve its smoothness and time allocation, which can further reduce the exploration time.

We evaluate FSMP by comparing it with existing popular methods in simulations. Furthermore, the proposed planner is successfully integrated into a fully autonomous MAV, and its performance is validated in real-world environments. The comparative results demonstrate that our method outperforms the existing ones. The main advantages and contributions of this work are summarized as follows.

1) We propose a frontier-sampling-mixed planner for fast autonomous exploration of complex and large 3-D environments, which can overcome the shortcomings of frontier- and sampling-based methods and achieve real-time performance onboard an MAV.
2) To speed up the frontier detection, F^3D with the guarantee of completeness and soundness is elaborately designed. F^3D can identify frontiers quickly and, thus, facilitate the following exploration process.
3) To achieve a complete exploration of the environment, we propose to incrementally build and maintain a road map using uniformly deterministic sampling. The road map can return a feasible path toward each exploration candidate directly and efficiently.
4) We propose a two-stage path planner to rapidly compute the global optimal exploration path on the road map using the devised lazy evaluation strategy and then smooth the path for further improving the exploration efficiency.

The remainder of this article is organized as follows. In Section II, we review related work on autonomous exploration in three aspects. We define notions and formulate the exploration problem in Section III. In Section IV, we first present an overview of our framework and then introduce the proposed exploration planner in detail. We evaluate the performance of our method comprehensively in Section V. Finally, we conclude this article and outline the future work in Section VI.

II. Related Work

A. Frontier-Based Exploration Algorithms

The frontier-based exploration is the most classic strategy for robotic exploration, and it was first proposed by Yamauchi [10]. The frontier is a boundary, which separates the known areas from those unknown in the environment. Commanding the robot to the frontier region can observe new areas and, thus, complete the exploration. Therefore, frontier detection is the key component of frontier-based exploration algorithms [10], [11], [12], [13], [19]. The naive frontier detection method [10] needs to process the whole map data every time it is invoked, which is inefficient, especially in 3-D scenarios. Keidar and Kaminka [20] proposed two promising algorithms for frontier detection, i.e., wavefront frontier detector (WFD) and fast frontier detector (FFD). Instead of processing the whole map data in every iteration, WFD examines the known part of the map, which can effectively reduce the computational overhead. However, WFD will degenerate into the naive one when the map is extended. Different from WFD, FFD only examines the contour that consists of the sensor readings. Even though FFD can be executed fast, we observe it has two disadvantages. First, FFD should be invoked after each scan, which may waste calculations. Second, FFD is a sensor-based method, and it is only suitable for 2-D lasers. Similar to FFD, Quin et al. [21] proposed the frontier-tracing frontier detection (FTFD) algorithm that only needs to examine the perimeter of the FOV of the sensor after each scan. Quin et al. [22] proposed the expanding-WFD (EWFD), which just needs to search newly observed regions based on the assumption that the entropy of each grid cell can only decrease. However, the assumption will not hold when there are moving obstacles in the environment or the sensor has a nonnegligible noise. To speed up frontier detection, the axis-aligned bounding box (AABB) is used in [13]. Specifically, the AABB of the observations of the sensor will be recorded. Once a free cell in the AABB is found, the region growing (RG) algorithm will be invoked for detecting new frontier cells. Nevertheless, it still scans unnecessary areas that are lying within the AABB.

B. Sampling-Based Exploration Algorithms

For sampling-based autonomous exploration [6], [8], [9], [23], it starts by randomly sampling exploration candidates (also known as viewpoints) in the free space of the environment. Then, the viewpoints will be evaluated for the determination of the exploration target. Inspired by the next-best-view (NBV) approaches [24], Bircher et al. [6] proposed the NBV planner (NBVP), which runs in a receding horizon fashion. NBVP utilizes the rapid-exploring random tree (RRT) to generate random viewpoints and only executes the first segment of the best branch in every iteration. It is proven that NBVP is efficient for exploring simple individual regions. Nevertheless, when the environment is complex and composed of multiple subregions, NBVP will struggle in a local narrow area or terminate prematurely. To avoid getting stuck in the local area, Schmid et al. [8] proposed an informative path planner (IPP) to build a large-size single tree during exploration, which can capture the global information of the environment and, thus, avoid getting stuck in a local area. Yet, the major limitation of this method is that it needs a lot of computational resources to maintain the large tree. Different from the above methods, the work proposed in [9] only maintains the history trajectory that the robot has executed in earlier iterations. When the robot has fully explored its vicinity area, it will trace back the history trajectory to find a valid exploration target. Tracing back previous trajectories is effective in avoiding getting stuck in local minima, but it is not efficient for the robot to transit to different regions.

C. Hybrid Frontier- and Sampling-Based Algorithms

There are also algorithms that incorporate frontier- and sampling-based methods into a unified framework [15], [16], [17], [18]. Selin et al. [15] proposed the autonomous

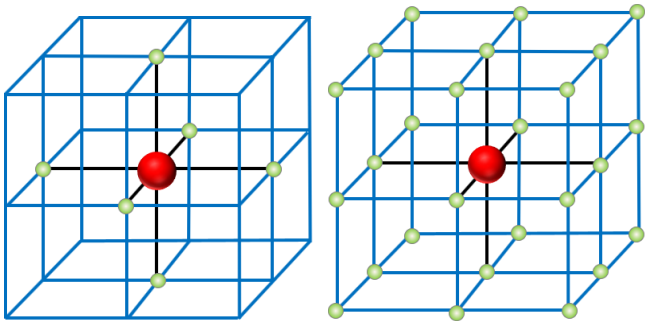
exploration planner (AEP), which utilizes NBVP for local exploration and the frontier-based method for global exploration. When the robot gets stuck, the frontiers are used to direct the robot to global regions. Nevertheless, AEP only caches the location of the frontiers, and a separate path planner is required for computing the global exploration path. Similar to [15], the work in [16] proposed a hybrid exploration framework that combines frontier- and sampling-based methods. In their framework, the hybrid planner first extracts the frontiers of the environment map. Then, exploration candidates are sampled from the map frontiers and evaluated to determine the next best view. Wang et al. [17] proposed to incrementally build a road map and extract frontiers around the nodes of the road map. The information gain and the cost-to-go of each exploration candidate can be queried on the road map efficiently. However, the road map needs to be extended after each scan, and there is no completeness guarantee for the frontier detection algorithm. Respall et al. [23] proposed a history-ware exploration planner. The method maintains a graph structure that consists of the old viewpoints from earlier iterations. Once the robot gets stuck in the dead end of the environment, it will reseed the active nodes on the graph to find a collision-free path toward global frontiers.

In summary, both frontier- and sampling-based exploration methods have their inherent disadvantages. It is intuitive to combine these two strategies in a unified framework for improving the efficiency of robotic exploration. Existing methods, however, either achieve this combination loosely or make decisions greedily in the local context. Therefore, we propose a novel frontier-sampling-mixed planner for fast exploration of complex and large 3-D environments with an MAV.

III. Problem Formulation

A. Definitions

1) *Volumetric Map:* A 3-D occupancy grid-based representation of the environment $\mathcal{V} \subset \mathbb{R}^3$, i.e., $\mathcal{M}$, where each grid cell in the volumetric map is named a voxel v, $v \in \mathcal{V}$.
2) *Unknown Space:* A subspace of $\mathcal{V}$ that has not been covered yet by the onboard sensor of the robot, i.e., $\mathcal{V}_{\text{un}}$ ($\mathcal{V}_{\text{un}} \subset \mathcal{V}$).
3) *Free Space:* A subspace of $\mathcal{V}$ that has already been covered by the onboard sensor. It is not occupied by any obstacles, i.e., $\mathcal{V}_{\text{free}}$ ($\mathcal{V}_{\text{free}} \subset \mathcal{V}$).
4) *Occupied Space:* A subspace of $\mathcal{V}$ that has already been covered by the onboard sensor. It is occupied by obstacles, i.e., $\mathcal{V}_{\text{occ}}$ ($\mathcal{V}_{\text{occ}} \subset \mathcal{V}$).
5) *Frontier Voxel:* A free voxel $v_f \in \mathcal{V}_{\text{free}}$ of the Volumetric map that has at least one unobserved voxel $v_{\text{un}} \in \mathcal{V}_{\text{un}}$ as its neighbor. As shown in Fig. 1(a), we designate the connectivity of voxels as six-connected in this case.
6) *Frontier:* A set of connected frontier voxels, i.e., f, $\text{f} = \{v_f \in \mathcal{V}_{\text{free}} : \exists \text{ neighbor}(v_f) \in \mathcal{V}_{\text{un}}\}$. In this case, the connectivity of frontier voxels is 26 [see Fig. 1(b)].
7) *Region of Interest (ROI):* A subspace of $\mathcal{V}$ that the frontier detector needs to search for detecting frontiers.



(a) (b)

Fig. 1. Illustration of the adjacency relationship of the map voxels. The red sphere indicates a specific voxel, and the green spheres denote its neighbors. (a) Six-connected. (b) 26-connected.

8) *Road Map:* An undirected graph $\mathcal{R}$ that consists of nodes N and edges E, i.e., $\mathcal{R} = \{N, E\}$. Essentially, the graph $\mathcal{R}$ is a sparse skeleton representation of $\mathcal{V}_{\text{free}}$.
9) *Exploration Candidate:* A node of the road map that implies the location of informative regions needed to be explored.

B. Problem Statement

In this article, we investigate the problem of autonomous exploration of 3-D environments with an MAV. The environment will be an unknown but spatially bounded space $\mathcal{V} \subset \mathbb{R}^3$. We assume that the MAV is equipped with a forward-looking depth sensor such as an RGB-D camera, which usually has a limited FOV. Let the state of the MAV be $\text{x} := \{p, \xi\} \in \mathcal{X} \subset \mathbb{R}^4$, where $\mathcal{X}$ is the configuration space, $p = \{x, y, z\} \in \mathbb{R}^3$ is the position and $\xi \in \mathbb{R}^1$ is the yaw angle. At state x, the MAV will collect its observation $O(\text{x})$ and integrate it into a probabilistic volumetric map $\mathcal{M}$ [25]. This map representation divides $\mathcal{V}$ into three subspace: $\mathcal{V}_{\text{un}}$, $\mathcal{V}_{\text{free}}$, and $\mathcal{V}_{\text{occ}}$.

Initially, we have $\mathcal{V}_{\text{un}} = \mathcal{V}$. With the progress of autonomous exploration, it becomes $\mathcal{V}_{\text{un}} = \mathcal{V} \setminus \{\mathcal{V}_{\text{free}} \cup \mathcal{V}_{\text{occ}}\}$. Our ultimate objective is to explore the unknown space in the environment as much as possible, i.e., let $\mathcal{V}_{\text{un}} \to \emptyset$. To achieve the objective more efficiently, we aim to maximize the utility function $\mathcal{U}(\cdot)$ when computing the exploration path in each decision-making epoch, as follows:

$$\max \mathcal{U}(\mathcal{L}(\gamma),\ \mathcal{I}(\gamma)) \quad \text{s.t.} \begin{cases} \gamma \subset \mathcal{V}_{\text{free}} \\ g(\gamma) \geq 0 \end{cases} \tag{1}$$

where $\gamma : [0, 1] \to \mathcal{X}$ denotes the exploration path and it is defined as a sequence of states of the MAV, $\mathcal{L}(\gamma)$ is the motion cost for the robot following the exploration path, $\mathcal{I}(\gamma)$ is the expected exploration gain that the MAV can obtain along the path, and $g(\gamma) \geq 0$ denotes the planned path should meet designated constraints. We can learn that $\mathcal{U}(\cdot)$ takes as input parameters the motion cost $\mathcal{L}(\gamma)$ and the expected exploration gain $\mathcal{I}(\gamma)$ and returns an evaluation result. The path that maximizes $\mathcal{U}$ will be executed to explore the environment.

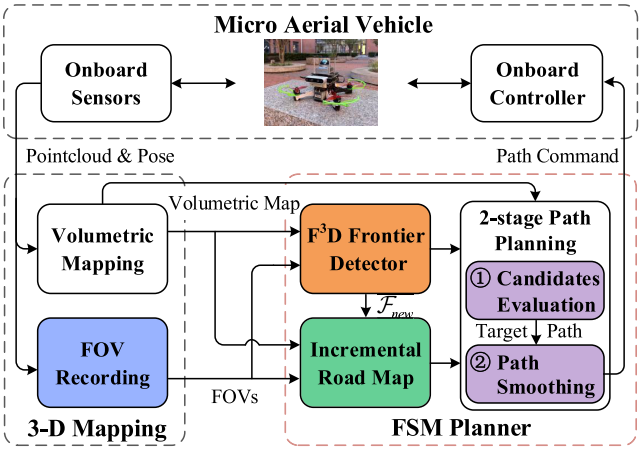


Fig. 2. Overall system architecture of the proposed exploration framework.

IV. Methodology

A. Framework Overview

The overall system architecture of our exploration framework is shown in Fig. 2. The 3-D mapping module integrates the depth measurement and pose estimation into the volumetric map $\mathcal{M}$ while recording the FOVs of the onboard sensor. Whenever FSMP is invoked, the frontier detector first extracts new frontiers and eliminates the ones covered by the sensor (see Section IV-B). After that, the incremental road map is extended, which can return a feasible path from the current position of the robot to any exploration candidate with its corresponding motion cost (see Section IV-C). Exploration candidates are evaluated, and the one with the maximum exploration utility is selected as the next exploration target. The path toward the best exploration target will be further optimized for improving the efficiency (see Section IV-D). Finally, the best path is subscribed by the controller to command the MAV for accomplishing the exploration mission. The task will be terminated when there is no frontier exists.

B. Fast FOV-Based Frontier Detection

In this section, we propose our F^{3}D, which is an incremental frontier detector that only needs to examine a very minimal subspace of the whole environment. Since, in the frontier searching process, only the information of the FOV of the onboard sensor is required for determining the ROI, F^{3}D is general for both 2-D and 3-D scenarios. In addition, F^{3}D is also not confined by the specific sensor type (e.g., laser range finders, sonars, and depth cameras) since the FOV of any type of sensor can be defined in advance. Therefore, if the sensor type is determined, the ROI required by F^{3}D can be computed directly.

As illustrated in Fig. 3, every time the exploration path is performed by the MAV, the FOVs of its onboard sensor are recorded. Furthermore, when one frontier is detected, the AABB of this frontier is also computed and recorded. Once F^{3}D is invoked, we first extract the FOVs of the onboard sensor and store them in $\mathcal{S}_t$. Due to the map updating event only occurring in the ROI that consists of the FOVs, only the observations in the ROI can affect the frontier detection.

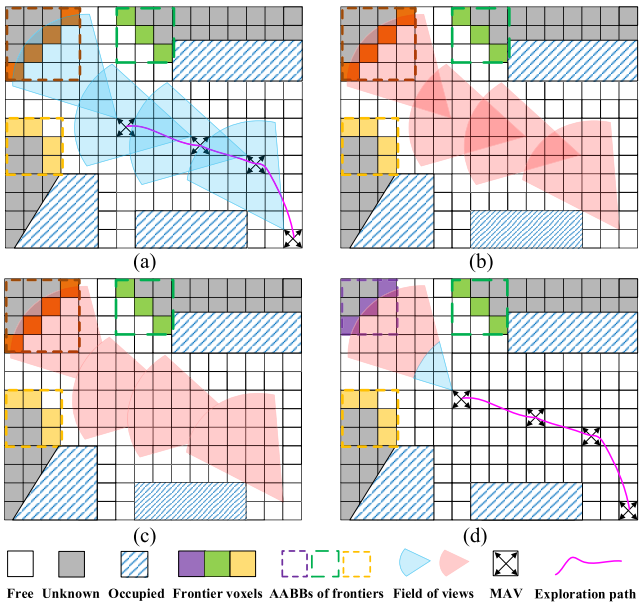


Fig. 3. 2-D illustration of the detection process of F^{3}D. (a) MAV is performing the exploration path; meanwhile, the FOVs of its onboard sensor are recorded. (b) There are overlapping areas between the FOVs. (c) F^{3}D can avoid repeatedly examining the voxels of the overlapping areas. (d) In this case, new frontiers are detected in the last recorded FOV.

Therefore, the frontier detector can be constrained to the ROI to maximize efficiency. As mentioned in [21], the robot cannot enter unknown space, and all the known regions of the map are connected naturally. When the sensor covers unknown areas, the FOV of the sensor must span both known and unknown space [see Fig. 3(a)], i.e., the FOV of the sensor must be intersecting with frontiers. Consequently, the old frontiers who are intersect with the FOV of the onboard sensor should be removed from the frontier database (i.e., $\mathcal{F}_{t-1}$) and reexamined when searching for new frontiers (see Algorithm 1, lines 3–11 and lines 33–39). If they are still frontiers, they will be stored in $\mathcal{F}_t$. Otherwise, they will be marked as nonfrontier voxels. Furthermore, when there are dynamic objects or the sensor data has a nonnegligible noise, new frontiers will also appear in the location where no old frontiers exist. To guarantee the completeness of the frontier detection, F^{3}D will examine all voxels that are lying within the space covered by the sensor of the robot since the last call.

It can be seen in Fig. 3(b) that there are overlapping areas between the FOVs of the sensor, and this can cause inefficiencies in frontier detection especially when the map is updating frequently. In order to avoid rescanning the overlapping areas, F^{3}D maintains two sets in the frontier detection process. This strategy is inspired by the A* algorithm [26].

1) *frontierSet:* A set of voxels that have already been marked as valid frontier voxels (see Algorithm 1, lines 8, 21, and 37).
2) *closedSet:* A set of voxels lying within the ROI has already been examined by F^{3}D (see Algorithm 1, lines 2 and 29).

With the above two sets, F^{3}D can determine whether there is a need to examine a specific voxel. Eventually, F^{3}D can

Algorithm 1 Fast FOV-Based Frontier Detection

Input: *frontierSet*, $\mathcal{S}_t$, $\mathcal{F}_{t-1}$
Output: $\mathcal{F}_t$

1: *deleteSet* $\leftarrow \emptyset$;
2: *closedSet* $\leftarrow \emptyset$;
3: **for** each f $\in \mathcal{F}_{t-1}$ **do**
4: **if** f is intersecting with $\mathcal{S}_t$ **then**
5: **if** any *voxel* $\in$ f is changed **then**
6: *deleteSet* $\leftarrow$ *deleteSet* $\cup$ f;
7: $\mathcal{F}_{t-1} \leftarrow \mathcal{F}_{t-1} \setminus$ f;
8: *frontierSet* $\leftarrow$ *frontierSet* $\setminus$ f;
9: **end if**
10: **end if**
11: **end for**
12: // Search for new frontiers in the ROI
13: **for** each $s \in \mathcal{S}_t$ **do**
14: **for** each *voxel* $\in s$ **do**
15: **if** *voxel* $\in$ *closedSet* **then**
16: continue;
17: **end if**
18: **if** any *nbr* $\in$ *Neighbor*(*voxel*) belongs to a frontier $f_{nbr} \wedge$ any *voxel* $\in f_{nbr}$ is changed **then**
19: *deleteSet* $\leftarrow$ *deleteSet* $\cup$ f_{nbr};
20: $\mathcal{F}_{t-1} \leftarrow \mathcal{F}_{t-1} \setminus f_{nbr}$;
21: *frontierSet* $\leftarrow$ *frontierSet* $\setminus f_{nbr}$;
22: **end if**
23: **if** *voxel* is a new frontier voxel **then**
24: *newFrontier* $\leftarrow$ *Extract_Frontier*(*voxel*) (Algorithm 2);
25: $\mathcal{F}_{new} \leftarrow \mathcal{F}_{new} \cup$ *newFrontier*;
26: *frontierSet* $\leftarrow$ *frontierSet* $\cup$ *newFrontier*;
27: continue;
28: **end if**
29: mark *voxel* as *closedSet*;
30: **end for**
31: **end for**
32: // Recheck the frontier voxels in the *deleteSet*
33: **for** each *voxel* $\in$ *deleteSet* **do**
34: **if** *voxel* is a frontier voxel $\wedge$ *voxel* $\notin$ *frontierSet* **then**
35: *newFrontier* $\leftarrow$ *Extract_Frontier*(*voxel*) (Algorithm 2);
36: $\mathcal{F}_{new} \leftarrow \mathcal{F}_{new} \cup$ *newFrontier*;
37: *frontierSet* $\leftarrow$ *frontierSet* $\cup$ *newFrontier*;
38: **end if**
39: **end for**
40: $\mathcal{F}_t \leftarrow$ *Process_Frontier*($\mathcal{F}_{new}$, $\mathcal{F}_{t-1}$);

guarantee the searching region required by frontier detection is nonrepetitive, as the one shown in Fig. 3(c).

In one specific FOV, we utilize a BFS-based strategy for extracting and clustering new frontiers. First, the voxels in the FOV will be examined in turn to check if it is a frontier voxel (see Algorithm 1, lines 13–31). Once a frontier voxel is detected, the BFS algorithm will be invoked for frontier extraction (see Algorithm 1, lines 24 and 35). The BFS utilizes the new frontier voxel as a starting point to extract all frontier voxels that belong to one cluster based on the connectivity

Algorithm 2 BFS-Based Frontier Extraction

Input: *voxel* // the starting point of the BFS
Output: *newFrontier* // the set of frontier voxels that are belonging to the same frontier

1: *tempQueue* $\leftarrow \emptyset$;
2: *newFrontier* $\leftarrow \emptyset$;
3: ENQUEUE(*tempQueue*, *voxel*);
4: **while** *tempQueue* is not empty **do**
5: *voxel* $\leftarrow$ DEQUEUE(*tempQueue*);
6: **if** *voxel* $\in$ *closedSet* $\vee$ *voxel* $\in$ *frontierSet* **then**
7: continue;
8: **end if**
9: **if** *voxel* is a frontier voxel **then**
10: *newFrontier* $\leftarrow$ *newFrontier* $\cup$ *voxel*;
11: **for** each *nbr* $\in$ *Neighbors(voxel)* **do**
12: **if** *nbr* $\notin$ *closedSet* $\wedge$ *voxel* $\notin$ *frontierSet* $\wedge$ *voxel* is a frontier voxel **then**
13: ENQUEUE(*tempQueue*, *nbr*);
14: **end if**
15: **end for**
16: **end if**
17: **end while**
18: **return** *newFrontier*

definition (see Algorithm 2, lines 4–17). Consequently, the detected frontier voxels are grouped naturally, and no more independent clustering operation is required. The new frontier is stored in *newFrontier*, a database used to store the new frontier voxels temporarily (see Algorithm 2, lines 18). In general, the size of a frontier might be too large to make efficient decisions. Therefore, similar to [5] and [13], we perform an additional frontier process operation to split large frontiers into small clusters and compute their centroids by the function *Process_Frontier()* (see Algorithm 1, line 40). This operation is conducted when all new frontiers are detected.

As shown in Fig. 3(d), F^{3}D can detect frontiers completely and accurately with high efficiency, and it is not necessary to be executed after each scan. Therefore, the flexibility of the proposed frontier detector is satisfactory. In Section IV-E, we will prove that F^{3}D is complete and sound, and we will also analyze the computational complexity of F^{3}D.

C. Uniformly Deterministic Sampling-Based Incremental Road Map Construction

The road map is responsible for path finding in the exploration. Initially, the road map $\mathcal{R} = \{p_0, \emptyset\}$, where p_0 is the home position of the MAV. In our case, $\mathcal{R}$ needs to extend when new regions are covered by the sensor. Generally, random sampling strategies will be employed to sample a large space of the environment to construct the road map. However, random sampling tends to result in an uneven distribution of the road map or needs a lot of effort to obtain a sequence of evenly distributed nodes. Based on our tests, to achieve a similar distribution of nodes, the random sampling strategy typically requires sampling and evaluating several times more nodes than the deterministic sampling strategy. To better estimate the utility of the exploration candidates, the road

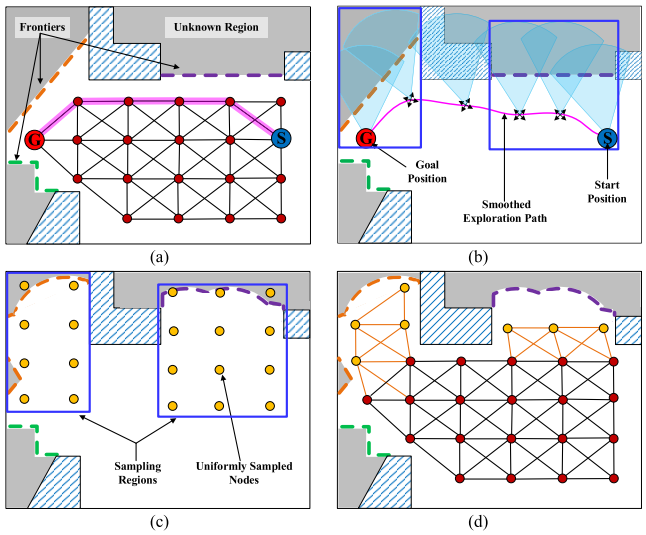


Fig. 4. 2-D schematic of our uniformly deterministic sampling-based incremental road map construction. (a) Exploration path can be found on the road map $\mathcal{R}$. (b) MAV follows the exploration path while recording its FOVs. (c) All the FOVs that intersect with the same new frontier are extracted and merged into one AABB (i.e., the blue rectangles). In that region, uniform sampling is conducted. (d) Feasible new nodes (the yellow circles) and corresponding edges (the brown lines) are added to $\mathcal{R}$.

map is required to be distributed in the free space of the environment comprehensively and evenly. This motivates us to propose an incremental road map construction algorithm that uniformly samples the new regions observed by the onboard sensor of the MAV.

As shown in Fig. 4, when the exploration path is determined, the MAV will follow this path to explore unknown regions of the environment. It is obvious that only the newly covered regions need to be sampled for sampling new road map nodes. If the sampling can be biased in these regions, the sampling efficiency will be improved. Fortunately, the FOVs of the onboard sensor are recorded in the frontier detection stage, and we will utilize them to determine the necessary sampling region (see Algorithm 3, lines 3–12). Specifically, we first extract the FOVs that are intersecting with the newly detected frontiers $\mathcal{F}_{\text{new}}$. In this process, the AABB of an FOV is computed to represent its effective region. Furthermore, if there are FOVs intersecting with each other, their AABBs will be merged into a larger one, e.g., the blue rectangles in Fig. 4(b) and (c) (see Algorithm 3, line 6). It is worth noting that our method can compute the sampling region adaptively instead of using a size-fixed and overestimated one.

In each sampling region, we utilize the Sukharev grid [27], which aims to achieve the most uniform distribution possible over the sampling region, to generate a set of new nodes. Specifically, a Sukharev grid is generated with the given resolutions $[l_x, l_y, l_z]$ in the x-, y-, and z-directions to cover each sampling region. After that, the sampling region will be partitioned into separated cells, and the centroids of these cells form the set of new nodes, $\mathcal{P}$. If one new node $p_{\text{new}} \in \mathcal{P}$ meets the following two conditions, it will be considered as a feasible node for constructing the road map, i.e.,

$$p_{\text{new}} \in V_{\text{free}} \tag{2}$$

$$d_{\min} \leq D(p_{\text{new}}, p) \leq d_{\max} \quad \forall\, p \in \mathcal{R} \tag{3}$$

Algorithm 3 Incremental Road Map Construction

Input: $\mathcal{V}_{free}$, $\mathcal{S}_t$, $\mathcal{F}_{new}$ // $\mathcal{S}_t$ and $\mathcal{F}_{new}$ are from Algorithm 1
Output: $\mathcal{R}$ // the road map

1: $\mathcal{S}_{temp} \leftarrow \emptyset$;
2: // Sampling regions determination
3: **for** each $s \in \mathcal{S}_t$ **do**
4: **if** s is intersecting with f **then** // $\forall$ f $\in \mathcal{F}_{new}$
5: **if** s is intersecting with $\hat{s}$ **then** // $\forall\, \hat{s} \in \mathcal{S}_{temp}$
6: $\hat{s} \leftarrow \hat{s} \cup s$;
7: $\mathcal{S}_{temp} \leftarrow \mathcal{S}_{temp} \cup \hat{s}$;
8: continue;
9: **end if**
10: $\mathcal{S}_{temp} \leftarrow \mathcal{S}_{temp} \cup s$;
11: **end if**
12: **end for**
13: // Road map construction
14: **for** each $s \in \mathcal{S}_{temp}$ **do**
15: $\mathcal{P} \leftarrow Uniform\_Sampling(s)$;
16: **for** each feasible $p \in \mathcal{P}$ **do**
17: **if** $\|p - p_i\| < d_{\min}$ **then** // $\forall\, p_i \in \mathcal{R}$
18: continue;
19: **end if**
20: $\mathcal{P}_{near} \leftarrow Near(p, \mathcal{R}, d_{\max})$;
21: **for** each $p_{near} \in \mathcal{P}_{near}$ **do**
22: **if** $No\_Collision(p,\ p_{near},\ \mathcal{V}_{free})$ and $||p - p_{near}|| < d_{\max}$ **then**
23: $E \leftarrow Connect(p,\ p_{near})$;
24: $\mathcal{R} \leftarrow \mathcal{R} \cup \{p, E\}$;
25: **end if**
26: **end for**
27: **end for**
28: **end for**

where p is an existing node of $\mathcal{R}$ and $D(p_{\text{new}}, p)$ is the Euclidean distance between p_{new} and p. $d_{\min}$ and $d_{\max}$ are two thresholds for restricting the edge length of the road map. The first condition (2) requires the new node to be lying within the free space. The second condition (3) ensures that the road map is not too dense and distributed evenly. Furthermore, when adding p_{new} to $\mathcal{R}$, the edge between p_{new} to the node $p \in \mathcal{R}$, $E(p_{\text{new}}, p)$, is also added if it is collision-free and meeting the condition stated in (3). The detail of the incremental road map construction process is shown in Algorithm 3, lines 14–28. Furthermore, to cope with dynamic or previously overlooked obstacles, we periodically check and prune the road map $\mathcal{R}$ to ensure that all the nodes and edges of $\mathcal{R}$ are collision-free.

After the road map is extended, we compute the nearest node p_{nearest} for each frontier f $\in \mathcal{F}_t$, and these nodes will be regarded as exploration candidates. Note that the candidate node is the one that can connect to its corresponding frontier without collision. With the road map, the path from the current position of the robot to the candidate and the corresponding motion cost can be queried efficiently. In what follows, we will show that one can achieve the global optimal decision-making based on the road map.

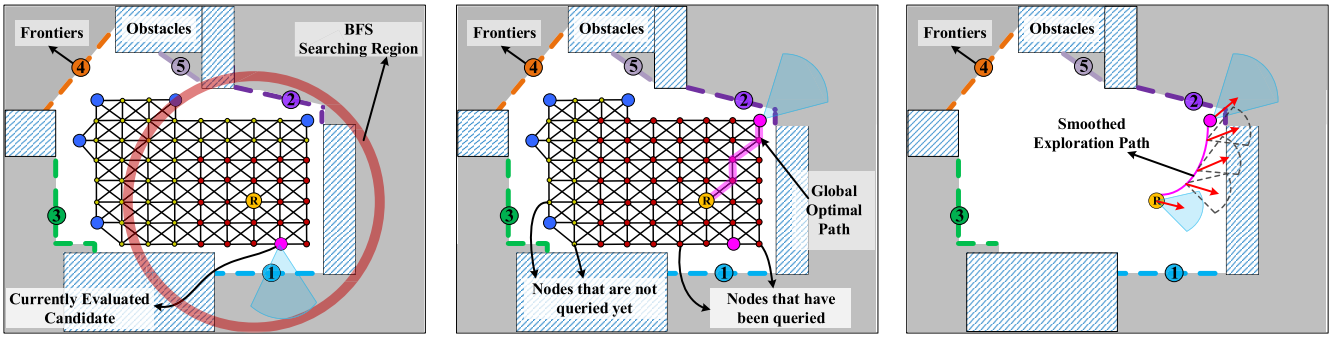


Fig. 5. 2-D illustration of our two-stage exploration path planning method. (a) When the Dijkstra algorithm finds a new exploration candidate (the purple solid circle), the new radius of Dijkstra's searching region (the region inside the red ring) will be computed. (b) Dijkstra algorithm continues searching outward and finds the next new exploration candidate (the purple solid circle at the top right corner). It is obvious that this new candidate has the maximum exploration gain, and thus, no further searching is necessary. The reason is that the remaining candidates cannot achieve a higher utility than this new candidate since they are guaranteed to have a higher motion cost than this new candidate. (c) Global optimal exploration path is obtained, and we perform a path smoothing technique to facilitate the MAV's tracking.

D. Two-Stage Exploration Path Planning

The path $\gamma(\mathrm{x}_r, \mathrm{x}_i)$ from the current state x_r of the MAV to any exploration candidate x_i can be queried efficiently on the road map $\mathcal{R}$ using a graph search algorithm, e.g., the Dijkstra algorithm. In addition, the utility of the candidate is typically computed by considering both the exploration gain and the motion cost [28]. In this section, we propose a two-stage planner that computes the global optimal exploration path first and then improves its smoothness and time allocation.

1) Global Optimal Exploration Path Planning: In the first stage, we evaluate all exploration candidates by considering their exploration gains and motion costs simultaneously. The utility function of (1) is formulated as follows:

$$\mathcal{U}(\mathcal{L}(\gamma),\ \mathcal{I}(\gamma)) = \mathcal{I}(\gamma(\mathrm{x}_r, \mathrm{x}_i))e^{-\mathcal{L}(\gamma(\mathrm{x}_r,\mathrm{x}_i))} \tag{4}$$

where the exploration gain $\mathcal{I}$ can be computed by

$$I(\gamma(\mathrm{x}_r, \mathrm{x}_i)) = \begin{cases} \mathrm{vGain}(\mathrm{x}_i) \\ \ \ \mathrm{or} \\ \mathrm{iGain}(\mathrm{x}_i) \end{cases} \tag{5}$$

where vGain() is the voxel gain and iGain() is the information gain. To maximize the exploration gain of each specific exploration candidate $\mathrm{x}_i := \{p_i, \xi_i\}$, we determine the yaw angle ξ_i of the MAV at p_i as the one maximizing $\mathcal{I}$ by using a yaw optimization method similar to [13].

Remark 1: For the exploration candidate, one can evaluate its exploration gain by calculating either the voxel gain or information gain. The voxel gain [6], [15], vGain(), is the total number (or volume) of unknown voxels that can be observed at the exploration candidate. The information gain [29], [30], iGain(), is the mutual information that the robot can obtain at the exploration candidate. In this work, we employ vGain() to compute the exploration gain of one candidate, as it is more efficient than computing the information gain iGain().

The motion cost for the MAV following the exploration path is formulated as follows:

$$\mathcal{L}(\gamma(\mathrm{x}_r, \mathrm{x}_i)) = \lambda \mathrm{dist}(\gamma(\mathrm{x}_r, \mathrm{x}_i)) \tag{6}$$

where dist() denotes the path length queried on the road map and $\lambda \geq 0$ is a tunable parameter.

To achieve global optimal planning, it is intuitive to evaluate the utility of all candidates and then choose the best one as the exploration target. However, computing the exploration gain and motion cost for all candidates typically results in a high computational overhead, which may prevent the application of our method in complex and large 3-D environments. Therefore, we propose a lazy evaluation strategy that allows us to obtain the global optimal exploration path while only evaluating a subset of all candidates. As shown in Fig. 5, we utilize the Dijkstra algorithm to search paths between the robot's current state x_r and the exploration candidates on the road map. Recall that when the onboard sensor is determined, its FOV can be defined in advance. Therefore, the maximum attainable exploration gain at any candidate state x will not exceed a bounded constant value $I_{\max}$. With this fact in mind, we can continually reduce the radius of Dijkstra's searching region when evaluating candidates, thereby lowering the computational cost without compromising global optimality.

Given the currently evaluated candidate x_i, we can compute its utility $\mathcal{U}_i$ by using (4). Let us assume that a candidate x_j to be evaluated has the maximum attainable exploration gain $I_{\max}$. Then, if we want $\mathcal{U}_j \geq \mathcal{U}_i$, the maximum motion cost of x_j should meet the following condition:

$$\mathcal{L}_j^{\max} \leq -\frac{1}{\lambda} \ln\left(\frac{\mathcal{U}_i}{I_{\max}}\right) \tag{7}$$

where we always have $\mathcal{U}_i = \mathcal{I}_i e^{-\mathcal{L}_i} \leq I_{\max}$, which can guarantee that $\mathcal{L}_j^{\max}$ is valid, and $\mathcal{L}_j^{\max}$ will be set as the new radius of Dijkstra's searching region.

At the beginning of each planning iteration, we first set the searching region as $\mathcal{L}^{\max} = +\infty$. As shown in Fig. 5(a), every time the Dijkstra algorithm searches a new candidate $\mathrm{x}_{\mathrm{new}}$, we first compute its utility $\mathcal{U}_{\mathrm{new}}$ and the corresponding maximum motion cost $\mathcal{L}_{\mathrm{new}}^{\max}$. If $\mathcal{U}_{\mathrm{new}}$ is larger than the utility of the previous best candidate $\mathrm{x}_{\mathrm{old}}$, we replace $\mathrm{x}_{\mathrm{old}}$ by $\mathrm{x}_{\mathrm{new}}$. Otherwise, the Dijkstra algorithm will continue searching for the next candidate [see Fig. 5(b)]. The above process is performed iteratively until the Dijkstra algorithm reaches the

region boundary and, thus, stops the search. After that, we can obtain the global optimal exploration goal x_g and the shortest path toward it. Practically, Dijkstra's searching region will be reduced dramatically after evaluating several candidates, resulting in a significant improvement in efficiency.

2) Path Smoothing: In the second stage, we will refine the optimal exploration path that is directly queried on the road map. It can be seen in Fig. 5(b) that the global optimal exploration path may be tortuous and, hence, not appropriate for robot tracking. To improve the smoothness of the path, we utilize a path smoothing algorithm similar to [31] that can compute the smoothest path while ensuring its clearance for safe flight in narrow spaces. As shown in Fig. 5(c), the smoothed exploration path will be more appropriate for tracking. In addition, to further speed up the exploration process, a numerical integration-based time-optimal velocity planning algorithm [32] is employed to generate the optimal time allocation along the smoothed exploration path.

E. Theoretical Analysis

1) F³D Completeness and Soundness: Similar to [20], we begin with a lemma, which demonstrates that F³D can always identify all new frontiers correctly (i.e., F³D will completely and correctly mark all new frontier voxels as frontier, which were not frontier voxels before current detection iteration). This lemma can help us prove that F³D is complete and sound.

Lemma 1: Suppose that v_f is a new frontier voxel at iteration t, which was not marked as a frontier voxel at any iteration τ, where $\tau < t$. Then, given a set of observations O_{t-1}^{t} that accumulated during iteration $t-1$ and iteration t, F³D will mark v_f as a frontier voxel.

Proof: Let O_{t-1}^{t} be the set of sensor observations accumulated during the period of two iterations t and $t-1$, and let the voxels covered by O_{t-1}^{t} be denoted by $\mathcal{V}(O_{t-1}^{t})$, i.e., the space that consists of O_{t-1}^{t}. At each iteration, only voxels in $\mathcal{V}(O_{t-1}^{t})$ will change their states. Furthermore, according to the frontier definition, only free voxels in $\mathcal{V}(O_{t-1}^{t})$ have the potential to be a frontier. F³D will examine every free voxel in $\mathcal{V}(O_{t-1}^{t})$ and mark it as a frontier if it is a valid frontier voxel (see Algorithm 1, lines 13–31). □

Based on Lemma 1, we can draw Theorem 1 as follows.

Theorem 1: Let v_f be a valid frontier voxel at iteration t. Then, F³D will mark v_f as a frontier voxel given the set of observations $\{O_0^1, \ldots, O_{t-1}^t\}$ that accumulated up until now.

Proof: In order to prove the completeness of F³D, there are two cases that need to be examined.

Case 1 (v_f Is a New Frontier Voxel at Iteration t): This case can be proved directly by Lemma 1.

Case 2 (v_f Was a New Frontier Voxel at Iteration τ, Where $\tau < t$): Let τ be the earliest iteration in which v_f was as a frontier. Based on Lemma 1, we know that v_f was marked as a frontier at that time. If v_f is not lying within the FOV of the onboard sensor recorded during the interval of iterations $t-1$ and t, which means that v_f has no chance to change its states because it was not covered yet by the sensor of the robot. If v_f is lying within one of the FOVs, it will be reexamined by F³D to confirm whether it is a frontier or not (see Algorithm 1, lines 3–11 and 33–39). Therefore, if v_f is still a frontier voxel at iteration t, it will be correctly marked by F³D. Furthermore, F³D will always maintain knowledge of the valid old frontiers from the time that they are identified. Consequently, v_f must be a frontier voxel that is maintained by F³D at iteration t.

The above two cases show that F³D will identify v_f to be a valid frontier voxel at iteration t. Consequently, we can say that Theorem 1 is true for any valid frontier voxel at iteration t, which follows that F³D is complete. □

In order to prove the soundness of F³D, we must demonstrate that there does not exist a case where F³D identifies a voxel as a frontier voxel when it is not.

Theorem 2: Let v be an arbitrary voxel at iteration t, which is not a frontier voxel. Then, F³D will not mark v as a frontier voxel given a set of observations $\{O_0^1, \ldots, O_{t-1}^t\}$.

Proof: Assume that v is an arbitrary voxel at iteration t, i.e., v is not a frontier voxel at iteration t. Then, according to our definition, v is either not a free voxel or v is a free voxel, but all of its six-connected neighbors are not unknown voxels. We have two cases that need to be considered:

Case 1 (v Was a Frontier Voxel at Iteration $t-1$): If v was a frontier voxel at iteration $t-1$, then it could be covered by the sensor (i.e., it might lying within one of the FOVs of the sensor). Therefore, v will be rechecked and removed from the frontier database (see Algorithm 1, lines 3–11, 33–39). However, there may be a rare case, i.e., when v is close to but outside the FOVs of the sensor and the sensor covered its only unknown neighbor. In this special case, v will not be a frontier voxel anymore. Fortunately, F³D will check the six-connected neighbors of each voxel inside the FOVs of the sensor, and thus, this special case can be handled (see Algorithm 1, lines 18–22).

Case 2 (v Was Not a Frontier Voxel at Iteration $t-1$): If v is not covered by O_{t-1}^{t}, then F³D will not scan it and, therefore, will not mark it as a frontier voxel. If v is a voxel that is covered by O_{t-1}^{t}, it will be checked and will not be marked as a frontier (see Algorithm 1, lines 13–31). □

2) Computational Complexity: For ease of computation, we use an AABB to estimate the region covered by each FOV of the sensor. Let $n \in \mathbb{Z}^+$ be the number of the recorded FOVs for frontier detection and $m \in \mathbb{Z}^+$ be the number of frontiers in $\mathcal{F}_{t-1}$. Since we use the standard C++ list template as the container to store frontiers, the time for removing one frontier from $\mathcal{F}_{t-1}$ is $\mathcal{O}(1)$. Therefore, lines 3–11 of Algorithm 1 run in $\mathcal{O}(mn)$ time. After that, F³D begins detecting new frontiers (see Algorithm 1, lines 13–26). Within each call to F³D, it scans every voxel in the recorded FOVs for frontier voxels. Let $k \in \mathbb{Z}^+$ be the total number of voxels in the ROI, and the time complexity for detecting new frontiers is $\mathcal{O}(6k)$ due to our connectivity definition. As can be seen in Fig. 6, the voxels that need to be checked for frontier detection by [13] and [20] are significantly more than ours. Intuitively, AABB + WFD scans fewer voxels than AABB + RG, and its time complexity should be supposed to be less than AABB + RG. However, according to the experimental results (see Section V), the conclusion is the opposite. This is because the AABB + WFD method maintains four container called *Map-Open-List*,

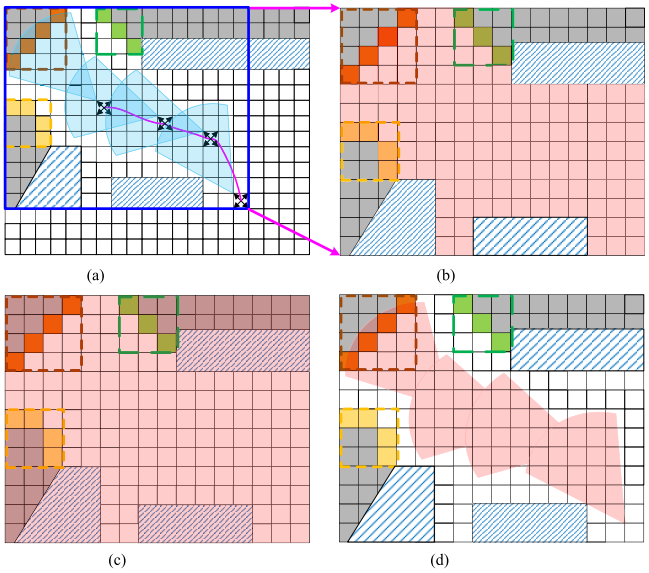


Fig. 6. Comparison of different frontier detection strategies. (a) Illustration of the AABB (the blue rectangle) of the sensor-covered region recorded in the duration of two iterations. (b) AABB + WFD method [20] scans all free voxels in the AABB. (c) AABB + RG method [13] scans all voxels in the AABB. (d) F^{3}D only scans voxels in the FOVs of the sensor.

Map-Close-List, *Frontier-Open-List*, and *Frontier-Close-List* to achieve nonrepetitive detection [20]. In contrast, our method only maintains two such containers, and the magnitude of data is less than AABB + WFD, i.e., only overlapping areas need to be handled. Then, if a voxel in the ROI is identified as a new frontier voxel, the BFS will be invoked to extract all voxels that belong to one frontier (see Algorithm 2). According to [33], we know that the complexity is linear in the size of the number of new frontier voxels, i.e., $\mathcal{O}\ (N(\text{new}-\text{frontier}-\text{voxels}))$. Also, let $h \in \mathbb{Z}^+$ be the voxels in *deleteSet*, and lines 28–34 of Algorithm 1 takes $\mathcal{O}\ (6h)$ time. Finally, the time complexity of F^{3}D is $\mathcal{O}\ (mn + 6(k+h) + N(\text{new}-\text{frontier}-\text{voxels}))$, which is close to the linear time complexity.

V. Evaluation Results

A. Implementation Details

In order to evaluate our method (FSMP) thoroughly, we conduct both simulation and real-world experiments in the context of an MAV. Fig. 7 shows the robot platforms used in the simulation and real world, respectively.

For the simulations, all algorithms are implemented by C++ on an OMEN9 SLIM laptop that runs Linux Ubuntu 20.04 LTS operation system with Intel Core i9-13900HX CPU at 5.4 GHz, 16-GB memory. We choose Gazebo [34] as the simulation engine since it provides realistic environments and robot models. The Firefly MAV provided by RotorS [35] [see Fig. 7(a)] is spawned in Gazebo to explore unknown 3-D environments. In our setup, the specifications of the VI-Sensor mounted on Firefly MAV have a sensing range of $[0.5, 5]$ m and a field of view of $[110, 90]^\circ$ in horizontal and vertical directions. Unless specified, otherwise, the rest of the parameters of our algorithm used in the simulation are listed in Table I.

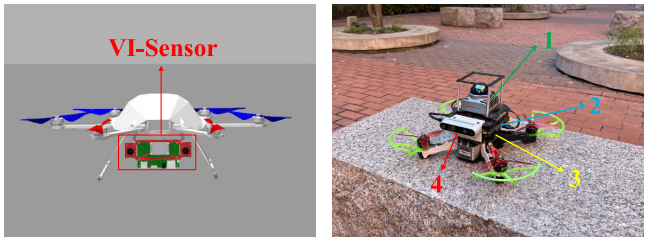


Fig. 7. Robot platforms that are used in the simulation and real-world experiments. (a) Simulated MAV, Firefly, is equipped with a forward-looking depth camera (i.e., VI-Sensor). (b) Real MAV is equipped with 1—a Livox Mid 360 LiDAR; 2—an onboard computer; 3—a PIXHAWK autopilot; and 4—a forward-looking depth camera.

TABLE I
Parameters Used in the Simulation and Experiment

Parameter	Value	Parameter	Value
d_{min}	0.5	l_x	0.8
d_{max}	1.5	l_y	0.8
λ	0.5	l_z	0.8

TABLE II
Computation Time of Different Frontier Detectors

Scenario	Computation Time (ms)					
	AABB+RG [13]		AABB+WFD [20]		F^{3}D (Ours)	
	Avg	Std	Avg	Std	Avg	Std
LargeMaze	15.5	11.7	53.1	51.6	**4.3**	**6.5**
CityBuilding	87.2	54.7	313.8	254.7	**38.9**	**35.6**
PowerPlant	55.3	42.6	155.0	168.9	**21.3**	**35.0**

For the real-world experiments, we run all algorithms on an Intel Core i7-1260P CPU. The custom-built MAV used in the real-world experiment is shown in Fig. 7(b). We localize the MAV by using a Livox Mid 360 LiDAR and build the volumetric map of the environment using data captured by the Intel Realsense D435i depth camera. The specifications of D435i have a sensing range of $[0.3, 5]$ m and a field of view of $[87, 58]^\circ$ in horizontal and vertical directions. The other parameters of the exploration algorithm used in the experiment are the same as in the simulations.

B. Simulation Study

In this section, we evaluate our proposed planner in three complex and large 3-D synthetic environments. As shown in Fig. 8, the environments involved in the simulation study are named *LargeMaze* [see Fig. 8(a)], *CityBuilding* [see Fig. 8(b)], and *PowerPlant* [see Fig. 8(c)], respectively.

First, to verify the efficiency of the proposed frontier detector, F^{3}D, we compare it with AABB + RG [13] and AABB + WFD [20] in the simulation trials. The average values and standard deviations of the computation time of different frontier detectors are shown in Table II. The results indicate that F^{3}D achieves a shorter time for frontier detection and smaller time variance in all environments. More specifically, F^{3}D is one order of magnitude faster than AABB + WFD and several times faster than AABB + RG. In addition, the performance of F^{3}D is consistently satisfactory in all environments, which indicates that F^{3}D has good scalability

(a) (b) (c)

Fig. 8. Three complex and large 3-D environments of increasing dimension and complexity are used in the simulation study. (a) *LargeMaze* environment is with the size of 40 × 40 × 3 m^3. (b) *CityBuilding* environment is with the size of 40 × 40 × 10 m^3. (c) *PowerPlant* environment is cropped to smaller dimensions (the part enclosed by the dotted box), and the size of the ROI is 31 × 31 × 26 m^3.

TABLE III
STATISTICAL RESULTS OF DIFFERENT METHODS WITH THE SAME SIMULATION CONFIGURATIONS

Scenario	Method	Exploration Time (s)				Traveling Distance (m)				Max Explored Volume (m^3)			
		Avg	Std	Max	Min	Avg	Std	Max	Min	Avg	Std	Max	Min
LargeMaze	IPP	1220.5	211.8	1420.5	1167.0	513.4	43.4	578.5	470.6	4533.6	113.5	4651.3	4372.0
	FUEL	469.5	39.1	517.5	460.0	406.6	25.4	436.4	369.5	**4755.8**	15.4	**4766.9**	**4728.6**
	FSample	692.0	51.9	735.5	666.0	461.3	**21.5**	486.2	443.9	4703.7	34.6	4750.2	4672.9
	Ours	**429.3**	**18.6**	**451.1**	**398.6**	**363.4**	22.6	**386.5**	**329.3**	4731.3	**5.5**	4741.0	4728.1
CityBuilding	IPP	-	-	-	-	-	-	-	-	11153.7	214.4	11366.6	10856.8
	FUEL	811.0	79.5	856.0	775.5	907.6	**51.9**	958.6	848.7	12356.7	166.0	12562.2	12188.3
	FSample	1114.5	323.7	1286.5	876.5	882.6	109.6	1010.6	728.5	12474.3	272.1	12752.9	12099.9
	Ours	**513.7**	**52.1**	**594.3**	**460.5**	**609.2**	90.4	**733.6**	**535.2**	**13349.8**	**119.0**	**13494.9**	**13167.2**
PowerPlant	IPP	-	-	-	-	-	-	-	-	16449.4	449.8	17161.5	16065.3
	FUEL	1539.8	130.5	1769.0	1403.5	1295.6	819.2	1376.3	1219.2	19707.8	228.3	20077.7	19518.3
	FSample	1740.0	138.3	1809.5	1626.0	1509.1	66.7	1564.9	1399.4	19827.4	352.7	20280.3	19466.7
	Ours	**686.1**	**16.8**	**709.7**	**662.2**	**791.2**	**32.1**	**827.4**	**748.6**	**21093.9**	**69.9**	**21169.4**	**20984.9**

−: denotes a method can not achieve 90% coverage of the environment.

in different environments. It is worth noting that all the above frontier detectors are complete and sound; therefore, the accuracy of these detected frontiers is identical according to our tests.

We also compare our planner, FSMP, with three existing exploration methods, which are tailored for MAVs equipped with depth cameras. All the methods are implemented using their open-source code adapted to specific simulated environments.

1) *IPP [8]:* A sampling-based method that consistently spans a single RRT* tree in the known free regions to achieve the global optimal exploration of the environment.
2) *FUEL [13]:* A frontier-based method that computes the optimal sequence to visit all frontiers by addressing TSP. After that, it computes the minimum-time trajectory for fast exploration.
3) *FSample [23]:* A hybrid method that combines the NBV sampling and frontier-based strategies in a unified framework to achieve global exploration.

Remark 2: It is worth noting that IPP is a pure sampling-based method, FUEL is a pure frontier-based method, and FSample is a hybrid method that combines both the sampling- and frontier-based ideas. The reason we chose these three methods to verify our method is that they are very popular algorithms, and each of them can be approximately viewed as an ablation case of our planner.

Note that in order to achieve a statistical comparison, we run all methods five times in each environment using the same initial configurations. In addition, we record the statistics following each run and summarize all the comparative results in Table III. These results include the exploration time when the explored volume reaches 90% of the total space, the traveling distance when the explored volume reaches 90% of the total space, and the max explored volume when the time limitation (set as three times of our method) is reached.

The first trial uses the MAV to explore the *LargeMaze* environment. This scenario is typically used to evaluate exploration methods, and we also use it to benchmark the performance of our method. In this trial, we set the map resolution to 0.2 m. The maximum linear velocity, the maximum linear acceleration, and the maximum angular velocity of the MAV are set to 1.0 m/s, 1.0 m/s^2, and 1.0 rad/s, respectively. Fig. 9(a) shows the volumetric mapping result of our method and the executed trajectories of each method. Table III gives the statistics of these methods in the *LargeMaze* environment. Our method can complete 90% exploration of the environment using the shortest traveling distance (363.4 m) and flight time (429.3 s) on average. As shown in Table III, our method can achieve 98.5% coverage of the environment on average when it stops exploring. Fig. 9(b) shows the average explored volume over the time of different methods. Our method improves the efficiency by more than 9.3% for the first 90% coverage compared to all other methods. It is worth noting that FUEL, which is known as the state-of-the-art in the exploration field,

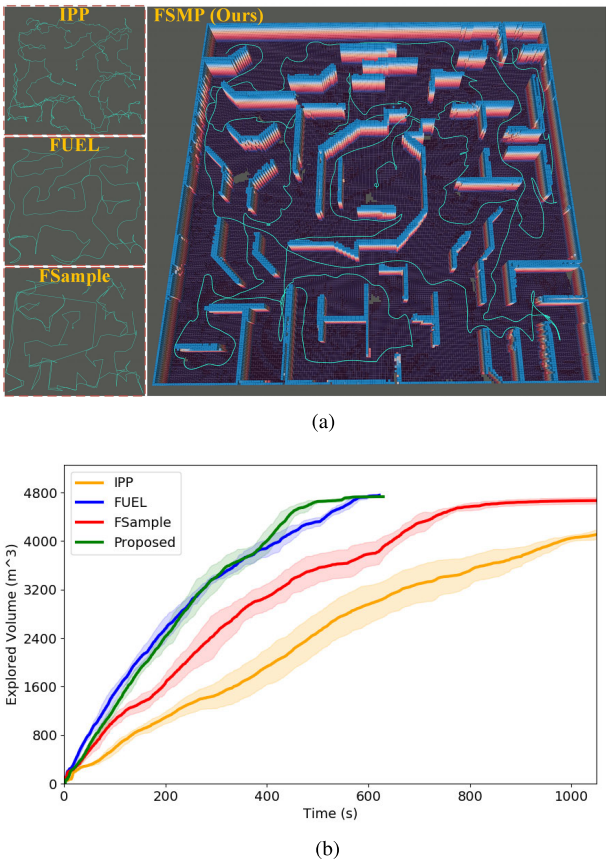


Fig. 9. Exploration results of the *LargeMaze* environment. (a) Volumetric mapping result of our method and the executed trajectories of IPP, FUEL, FSample, and the proposed method from their representative runs. (b) Exploration progress of these four methods.

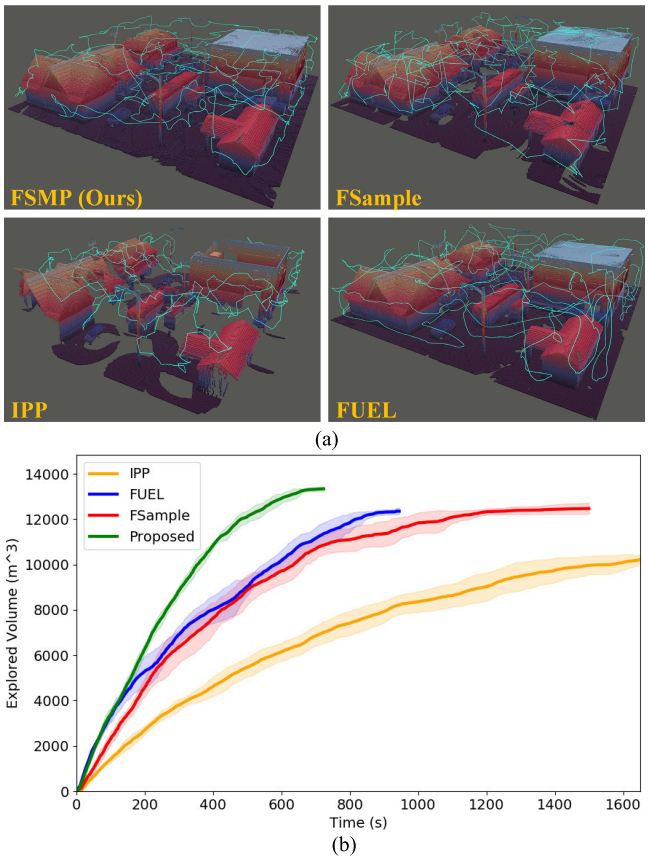


Fig. 10. Exploration results of the *CityBuilding* environment. (a) Volumetric mapping result of the environment and trajectories of IPP, FUEL, FSample, and the proposed method from their representative runs. (b) Exploration progress of these four methods.

TABLE IV
COMPUTATION TIME USED FOR EACH PLANNING ITERATION

Scenario	Method	Time (ms)			
		Avg	Std	Max	Min
LargeMaze	IPP	1171.5	370.3	3106.9	184.1
	FUEL	481.0	498.0	2491.0	4.1
	FSample	817.6	2287.0	16933.1	71.3
	Proposed	**22.2**	**12.7**	**94.1**	**0.22**
CityBuilding	IPP	1904.3	507.9	4162.9	485.2
	FUEL	625.7	492.0	3003.1	31.6
	FSample	675.5	1544.4	13336.0	97.5
	Proposed	**107.7**	**97.1**	**380.8**	**0.45**
PowerPlant	IPP	1677.79	1727.84	5618.2	173.8
	FUEL	1469.6	1727.8	15265.1	18.4
	FSample	629.6	2333.4	35037.0	4.3
	Proposed	**164.5**	**126.9**	**562.9**	**0.21**

demonstrated performance comparable to our method in this trial. The reason is that in this scenario, the number of frontiers is relatively small, and thus, the cost of solving TSP is low. The computational time used for each planning iteration of FUEL can be found in Table IV.

The second trial employs the MAV to explore the *CityBuilding* environment. Compared to *LargeMaze*, the *CityBuilding* scenario demonstrates complex geometrical changes along the z-axis, which can validate the basic performance of different exploration methods in real 3-D scenarios. In this trial, we set the map resolution to 0.1 m. The maximum linear velocity, the maximum linear acceleration, and the angular velocity of the MAV are set to 2.0 m/s, 2.0 m/s^2, and 2.0 rad/s, respectively. Fig. 10(a) shows the volumetric mapping results of the environment and the executed trajectories of the MAV. Fig. 10(b) shows the exploration progress of different methods over time. Table III gives the statistics of four methods in this scenario. It can be found that our method completes the first 90% coverage of the environment using 609.2 m of the travel distance and 513.7 s of flight time on average. Even though the second scenario is more complex and larger than *LargeMaze*, our method can achieve the largest coverage (98.9% on average) of the environment when it stops exploring. In addition, our method improves the exploration efficiency by more than 58.1% compared to the other methods. It can be found in Table IV that the computational time used for each planning iteration of FUEL is increased significantly. This is because, in large scenarios (especially open environments), the number of frontiers will increase dramatically, which will, thus, reduce the exploration efficiency of FUEL. In addition, the computational time of IPP is the longest, resulting in its inability to complete 90% coverage of the environment within the maximum time limitation.

The last trial uses the MAV to explore the *PowerPlant* environment. This scenario is the largest and most complex one of all scenarios. As shown in Fig. 8(c), PowerPlant features complex structures and narrow spaces to be explored. In this

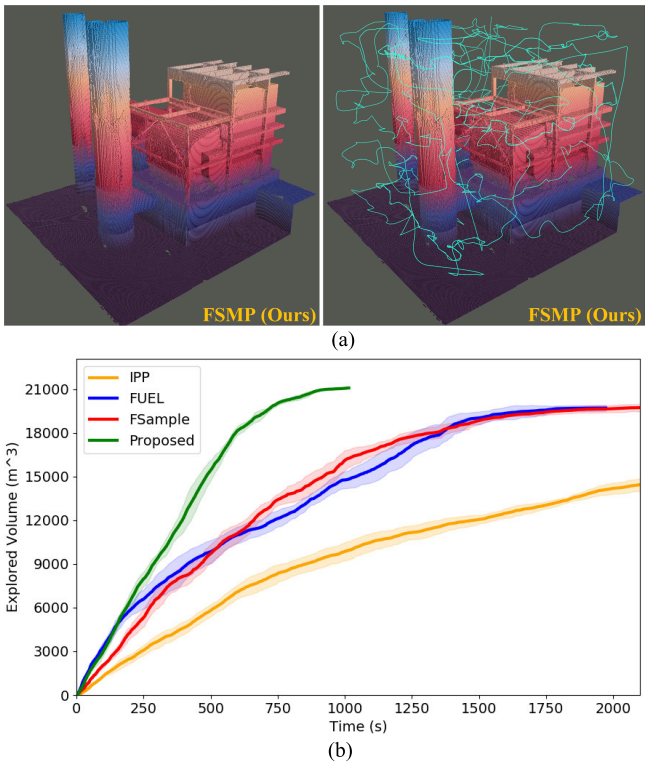


Fig. 11. Exploration results of the *PowerPlant* environment. (a) Volumetric mapping result and the executed trajectory of the proposed method from its representative run. (b) Exploration progress of four methods.

trial, we set the map resolution to 0.1 m. The maximum linear velocity, the maximum linear acceleration, and the angular velocity of the MAV are set to 2.0 m/s, 2.0 m/s^2, and 2.0 rad/s, respectively. Fig. 11(a) shows the volumetric mapping result and the executed trajectory of FSMP. Our method completes the first 90% coverage of the environment using 609.2 m of the travel distance and 791.2 s of flight time on average. Fig. 11(b) shows the explored volume over the time of four methods. It can be found in Table III that our method improves the exploration efficiency by more than 124.4% compared to the other methods. In addition, IPP, FUEL, and FSample cannot reach 95% coverage of the environment within the maximum time limitation. However, our method can achieve 98.1% coverage on average of the environment, demonstrating that our method possesses good scalability.

C. Real-World Experiments

To further validate our planner, we implement FSMP and FUEL, respectively, on a self-built quadrotor MAV [see Fig. 7(b)] to explore an outdoor environment with cluttered obstacles. As shown in Fig. 12(a), the environment to be explored is with a size of 20 × 26 × 3 m^3. For both FSMP and FUEL, the maximum linear velocity, the maximum linear acceleration, and the maximum angular velocity of the MAV are set to 2.0 m/s, 2.0 m/s^2, and 1.5 rad/s, respectively. The resolution of the volumetric map is set to 0.1 m. The LiDAR-based odometry, Point-LIO [36], is employed in the experiments for the MAV pose estimation. Note that there are no external devices used for real-world experiments, and

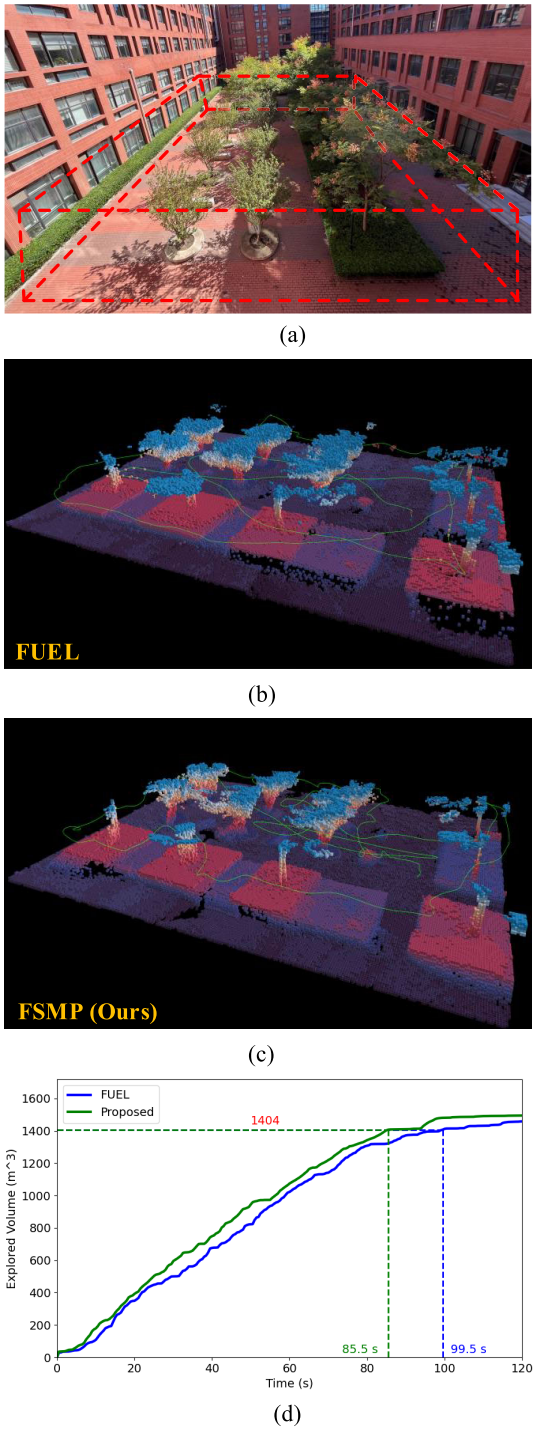


Fig. 12. Real-world experiment in an outdoor environment with cluttered obstacles. (a) Outdoor environment. (b) Exploration result of FUEL. (c) Exploration result of FSMP. (d) Exploration progresses.

all algorithms are running onboard the MAV in real time. To achieve fair evaluation, we set the maximum time used for exploration to 120 s.

The volumetric mapping results and the executed trajectories of FSMP and FUEL are shown in Fig. 12(b) and (c), respectively. The explored volume over time of FSMP and FUEL is shown in Fig. 12(d). It can be seen that our method can complete the first 90% coverage of the environment in

85.5 s, while FUEL takes 99.5 s, which is 16.3% longer than our method. In addition, FSMP can achieve 96.0% coverage (i.e., 1498.3 m^3 out of 1560 m^3) of the environment within the maximum time limit, whereas FUEL can only achieve 93.6% coverage (i.e., 1459.9 m^3 out of 1560 m^3).

The simulation and real-world results demonstrate the efficient and effective performance of our method in various environments. More details of all experiments are available in the Supplementary Video.

VI. Conclusion

In this article, a novel frontier-sampling-mixed exploration planner (FSMP) is developed by integrating the advantages of the frontier- and sampling-based strategies. According to the simulation studies, FSMP can achieve superior exploration performance over some state-of-the-art autonomous exploration methods in terms of exploration efficiency (more than 9.3% faster in the benchmark environment and 124.4% faster in the most complex and largest 3-D environment), computational time (several times faster at least), and explored volume (only our method achieved more than 98% coverage of the last 3-D environment). Finally, the real-world experiments demonstrated that our planner is effective for exploring large and 3-D environments and can be executed in real time onboard a self-developed MAV.

The main limitations of FSMP are twofold. First, we assume that the MAV can perfectly localize itself, as most methods do. However, state estimation errors are ubiquitous and cannot be ignored, especially in real-world environments. Large state estimation errors can significantly degrade the exploration performance or even lead to task failure. Second, we assume that the environment to be explored is static. However, in practical scenarios, dynamic obstacles are frequently encountered, leading to insufficient coverage. In our future work, we plan to consider odometry drifts in FSMP to improve its robustness under localization uncertainty and devise specific strategies for addressing exploration problems in dynamic environments. In addition, we are working on extending FSMP for fast exploration of large field environments with multiple MAVs.

References

[1] Z. Xu, B. Chen, X. Zhan, Y. Xiu, C. Suzuki, and K. Shimada, "A vision-based autonomous UAV inspection framework for unknown tunnel construction sites with dynamic obstacles," *IEEE Robot. Autom. Lett.*, vol. 8, no. 8, pp. 4983–4990, Aug. 2023.

[2] Y. Li, J. Wang, H. Chen, X. Jiang, and Y. Liu, "Object-aware view planning for autonomous 3-D model reconstruction of buildings using a mobile robot," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–15, 2023.

[3] Y. Liu, Z. Zhou, H. Sang, S. Yu, Y. Yan, and M. J. Er, "Efficient exploration of mobile robot based on DL-RRT and AP-BO," *IEEE Trans. Instrum. Meas.*, vol. 73, pp. 1–9, 2024.

[4] B. Zhou, H. Xu, and S. Shen, "Racer: Rapid collaborative exploration with a decentralized multi-UAV system," *IEEE Trans. Robot.*, vol. 39, no. 3, pp. 1816–1835, Jun. 2023.

[5] S. Zhang, X. Zhang, T. Li, J. Yuan, and Y. Fang, "Fast active aerial exploration for traversable path finding of ground robots in unknown environments," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–13, 2022.

[6] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon path planning for 3D exploration and surface inspection," *Auto. Robots*, vol. 42, no. 2, pp. 291–306, Feb. 2018.

[7] D. Duberg and P. Jensfelt, "UFOExplorer: Fast and scalable sampling-based exploration with a graph-based planning structure," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2487–2494, Apr. 2022.

[8] L. Schmid, M. Pantic, R. Khanna, L. Ott, R. Siegwart, and J. Nieto, "An efficient sampling-based method for online informative path planning in unknown environments," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1500–1507, Apr. 2020.

[9] X. Zhang, Y. Chu, Y. Liu, X. Zhang, and Y. Zhuang, "A novel informative autonomous exploration strategy with uniform sampling for quadrotors," *IEEE Trans. Ind. Electron.*, vol. 69, no. 12, pp. 13131–13140, Dec. 2022.

[10] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. Int. Symp. Comput. Intell. Robot. Automat. (CIRA)*, Jul. 1997, pp. 146–151.

[11] Y. Zhao, L. Yan, H. Xie, J. Dai, and P. Wei, "Autonomous exploration method for fast unknown environment mapping by using UAV equipped with limited FOV sensor," *IEEE Trans. Ind. Electron.*, vol. 71, no. 5, pp. 4933–4943, May 2024.

[12] J. Yu, H. Shen, J. Xu, and T. Zhang, "ECHO: An efficient heuristic viewpoint determination method on frontier-based autonomous exploration for quadrotors," *IEEE Robot. Autom. Lett.*, vol. 8, no. 8, pp. 5047–5054, Aug. 2023.

[13] B. Zhou, Y. Zhang, X. Chen, and S. Shen, "FUEL: Fast UAV exploration using incremental frontier structure and hierarchical planning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 779–786, Apr. 2021.

[14] X. Zhang, X. Xu, Y. Liu, H. Wang, X. Zhang, and Y. Zhuang, "FGIP: A frontier-guided informative planner for UAV exploration and reconstruction," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6155–6166, Apr. 2024.

[15] M. Selin, M. Tiger, D. Duberg, F. Heintz, and P. Jensfelt, "Efficient autonomous exploration planning of large-scale 3-D environments," *IEEE Robot. Autom. Lett.*, vol. 4, no. 2, pp. 1699–1706, Apr. 2019.

[16] A. Dai, S. Papatheodorou, N. Funk, D. Tzoumanikas, and S. Leutenegger, "Fast frontier-based information-driven autonomous exploration with an MAV," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2020, pp. 9570–9576.

[17] C. Wang, H. Ma, W. Chen, L. Liu, and M. Q.-H. Meng, "Efficient autonomous exploration with incrementally built topological map in 3-D environments," *IEEE Trans. Instrum. Meas.*, vol. 69, no. 12, pp. 9853–9865, Dec. 2020.

[18] H. Zhu, C. Cao, Y. Xia, S. Scherer, J. Zhang, and W. Wang, "DSVP: Dual-stage viewpoint planner for rapid exploration by dynamic expansion," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 7623–7630.

[19] A. Brunel, A. Bourki, C. Demonceaux, and O. Strauss, "SplatPlanner: Efficient autonomous exploration via permutohedral frontier filtering," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 608–615.

[20] M. Keidar and G. A. Kaminka, "Efficient frontier detection for robot exploration," *Int. J. Robot. Res.*, vol. 33, no. 2, pp. 215–236, Feb. 2014.

[21] P. Quin, D. D. K. Nguyen, T. L. Vu, A. Alempijevic, and G. Paul, "Approaches for efficiently detecting frontier cells in robotics exploration," *Frontiers Robot. AI*, vol. 8, Feb. 2021, Art. no. 616470.

[22] P. Quin, A. Alempijevic, G. Paul, and D. Liu, "Expanding wavefront frontier detection: An approach for efficiently detecting frontier cells," in *Proc. Australas. Conf. Robot. Automat.*, Jan. 2014, pp. 1–10.

[23] V. M. Respall, D. Devitt, R. Fedorenko, and A. Klimchik, "Fast sampling-based next-best-view exploration algorithm for a MAV," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 89–95.

[24] C. Connolly, "The determination of next best views," in *Proc. IEEE Int. Conf. Robot. Autom.*, Mar. 1985, pp. 432–435.

[25] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Auto. Robots*, vol. 34, no. 3, pp. 189–206, Apr. 2013.

[26] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. SCS-4, no. 2, pp. 100–107, Jul. 1968.

[27] L. Janson, B. Ichter, and M. Pavone, "Deterministic sampling-based motion planning: Optimality, complexity, and performance," *Int. J. Robot. Res.*, vol. 37, no. 1, pp. 46–61, Jan. 2018.

[28] M. Juliá, A. Gil, and O. Reinoso, "A comparison of path planning strategies for autonomous exploration and mapping of unknown environments," *Auto. Robots*, vol. 33, no. 4, pp. 427–444, Nov. 2012.

[29] C. Wang, W. Chi, Y. Sun, and M. Q.-H. Meng, "Autonomous robotic exploration by incremental road map construction," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 4, pp. 1720–1731, Oct. 2019.

[30] H. Gao, X. Zhang, J. Wen, J. Yuan, and Y. Fang, "Autonomous indoor exploration via polygon map construction and graph-based SLAM using directional endpoint features," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 4, pp. 1531–1542, Oct. 2019.

[31] T. Li, S. Zhang, X. Zhang, Q. Dong, and J. Huang, "SGS-planner: A skeleton-guided spatiotemporal motion planner for flight in constrained space," *IEEE/ASME Trans. Mechatronics*, vol. 30, no. 1, pp. 191–202, Feb. 2025.

[32] T. Kunz and M. Stilman, "Time-optimal trajectory generation for path following with bounded acceleration and velocity," in *Proc. Robot., Sci. Syst. VIII*, Jul. 2012, pp. 1–8.

[33] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, "Elementary graph algorithms," in *Introduction to Algorithms*. Cambridge, MA, USA: MIT Press, 2009.

[34] *Gazebo*. Accessed: Nov. 2024. [Online]. Available: http://gazebosim.org/

[35] F. Furrer, M. Burri, M. Achtelik, and R. Siegwart, "RotorS—A modular Gazebo MAV simulator framework," in *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Cham, Switzerland: Springer, 2016, pp. 595–625.

[36] D. He, W. Xu, N. Chen, F. Kong, C. Yuan, and F. Zhang, "Point-LIO: Robust high-bandwidth light detection and ranging inertial odometry," *Adv. Intell. Syst.*, vol. 5, no. 7, Jul. 2023, Art. no. 2200459.

Shiyong Zhang (Member, IEEE) received the B.Eng. degree in electrical engineering and automation from Northeast Forestry University, Harbin, China, in 2017, and the Ph.D. degree in control science and engineering from Nankai University, Tianjin, China, in 2022.

He is currently a Post-Doctoral Fellow with the Institute of Robotics and Automatic Information System (IRAIS) and also Tianjin Key Laboratory of Intelligent Robotics, Nankai University. His current research interests include autonomous aerial vehicles, autonomous exploration, informative path planning, and motion planning in complex environments.

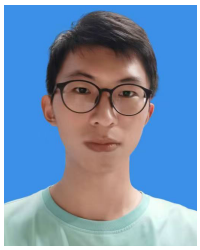
Xuebo Zhang (Senior Member, IEEE) received the B.Eng. degree in automation from Tianjin University, Tianjin, China, in 2006, and the Ph.D. degree in control theory and control engineering from Nankai University, Tianjin, in 2011.

From 2014 to 2015, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, University of Windsor, Windsor, ON, Canada. He was a Visiting Scholar with the Department of Mechanical and Biomedical Engineering, City University of Hong Kong, Hong Kong, in 2017. He is currently a Professor with the Institute of Robotics and Automatic Information System, Nankai University, and also with Tianjin Key Laboratory of Intelligent Robotics, Nankai University. His research interests include planning and control of autonomous robotics and mechatronic systems with a focus on time-optimal planning and visual servo control; intelligent perception including robot vision, visual servo networks, and SLAM; and reinforcement learning and game theory.

Dr. Zhang is a Technical Editor of IEEE/ASME TRANSACTIONS ON MECHATRONICS and an Associate Editor of *ASME Journal of Dynamic Systems, Measurement, and Control*.

Qianli Dong (Graduate Student Member, IEEE) received the B.Eng. degree in robot engineering from Northeastern University, Shenyang, China, in 2022. He is currently pursuing the Ph.D. degree in control science and engineering with the Institute of Robotics and Automatic Information System, Nankai University, Tianjin, China.

His current research interests include autonomous exploration, visual coverage, and motion planning in unknown environments.



Ziyu Wang received the B.Eng. degree in intelligent engineering and technology from Nankai University, Tianjin, China, in 2024, where he is currently pursuing the master's degree in control science and engineering with the Institute of Robotics and Automatic Information System.

His current research interests include autonomous aerial vehicles and motion planning in unknown environments.

Haobo Xi received the B.Eng. degree in artificial intelligence from Dalian University of Technology, Dalian, China, in 2023. He is currently pursuing the master's degree in artificial intelligence with the Institute of Robotics and Automatic Information System, Nankai University, Tianjin, China.

His current research interests include autonomous aerial vehicles and LiDAR-based simultaneous localization and mapping.

Jing Yuan (Member, IEEE) received the B.S. degree in automatic control and the Ph.D. degree in control theory and control engineering from Nankai University, Tianjin, China, in 2002 and 2007, respectively.

Since 2007, he has been with the College of Artificial Intelligence, Nankai University, where he is currently a Professor. His current research interests include robotic control, motion planning, and SLAM.